الكلية متعددة التخصصات - ورزازات
ⵜⴰⵢⵙⵉⵍⵜ ⵜⴰⵅⵜⵙⵙⵉⵙⵜ – ⵍⵓⵕⵣⵣⴰⵣⵜ
FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE

Reconnaissance des caractères Tifinagh par un réseau LeNet-5 implémenté from scratch

Youssef Khallouki
Master IAA
Encadrants : Pr. M. Benaddy, Pr. A. Hadri
Année universitaire 2024–2025
Code : `https://github.com/Ykhallouki25/Tp-lenet5-tifinagh`

Résumé

Nous implémentons en NumPy pur une version modifiée de LeNet-5 (7 couches, 63 661 paramètres) pour classer 33 caractères de l'alphabet Tifinagh (base AMHCD, 28 182 images) à partir d'images en niveaux de gris 32×32. Toutes les couches (convolution, pooling, fully connected, activations, perte softmax/entropie croisée) et les optimiseurs (SGD, Momentum, Nesterov, RMSprop, Adam) sont codés avec leurs passes avant et arrière, validées par différences finies (erreur relative $< 3 \times 10^{-7}$). Avec Adam, le réseau atteint **99,20 %** d'accuracy sur le jeu de test (4 224 images, 34 erreurs). Le code fournit aussi les courbes loss/accuracy, la matrice de confusion et les cartes de caractéristiques. Le code est disponible à l'adresse `https://github.com/Ykhallouki25/Tp-lenet5-tifinagh`.

1 Introduction

Le Tifinagh est l'écriture de la langue amazighe ; sa reconnaissance automatique est utile pour la numérisation de documents et l'éducation. Les réseaux convolutifs (CNN) sont l'approche de référence en reconnaissance de caractères depuis LeNet-5 [1], conçu pour les chiffres manuscrits. Dans ce travail, nous adaptons LeNet-5 à un problème à 33 classes et nous réimplémentons l'ensemble de la chaîne (propagation avant, rétropropagation, optimisation) sans bibliothèque d'apprentissage profond, afin de comprendre et de comparer les mécanismes sous-jacents. Les contributions sont : (i) une implémentation modulaire et paramétrable ; (ii) une comparaison de quatre optimiseurs ; (iii) des outils de visualisation ; (iv) des options d'amélioration (max-pooling, ReLU, dropout, L2, augmentation par translation). Le code complet (implémentation, scripts d'entraînement, notebook, résultats et figures) est disponible sur GitHub : `https://github.com/Ykhallouki25/Tp-lenet5-tifinagh`. La section 2 décrit les données, l'architecture, les formules mathématiques et le protocole ; la section 3 présente les résultats ; la section 4 les discute ; la section 5 conclut.

2 Méthodes

2.1 Données

Nous utilisons la base AMHCD (Amazigh Handwritten Character Database) : 33 classes de caractères Tifinagh, 854 images par classe (28 182 images au total, base parfaitement équilibrée), fournies en 64×64 niveaux de gris (trait clair sur fond sombre). Les images sont redimensionnées en 32×32, mises à l'échelle dans $[0, 1]$, puis standardisées avec la moyenne et l'écart-type de l'ensemble d'apprentissage. Le jeu est découpé de façon stratifiée par classe en 70 % (19 734 images d'apprentissage), 15 % (4 224 de validation) et 15 % (4 224 de test), avec une graine fixe (0).

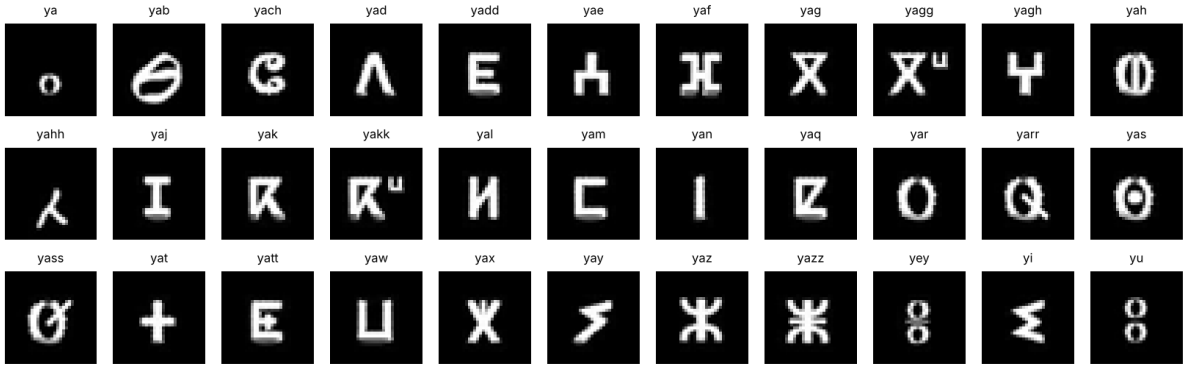


Figure 1: Un exemple par classe de la base (images 32×32 après redimensionnement ; noms des classes de la base).

2.2 Architecture

Le réseau suit le schéma : Entrée $1@32 \times 32 \to$ C1 (conv 5×5, 6 filtres) $\to$ S2 (pooling 2×2) $\to$ C3 (conv 5×5, 16 filtres) $\to$ S4 (pooling 2×2) $\to$ C5 (120 neurones) $\to$ F6 (84 neurones) $\to$ sortie (33 neurones).

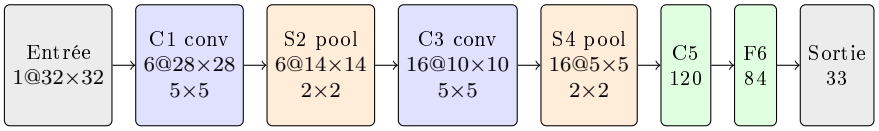


Figure 2: Architecture du LeNet-5 modifié (convolution en bleu, pooling en orange, fully connected en vert).

Couche	Sortie	Paramètres
C1	6@28 × 28	156
S2	6@14 × 14	0
C3	16@10 × 10	2 416
S4	16@5 × 5	0
C5	120	48 120
F6	84	10 164
Sortie	33	2 805
Total		63 661

Tableau 1: Dimensions et nombre de paramètres.

2.3 Formulation mathématique

Convolution. Pour une entrée $X \in \mathbb{R}^{C\times H\times W}$ et F filtres $W_f \in \mathbb{R}^{C\times k\times k}$ avec pas s et rembourrage p :

$$Y_{f,i,j} = b_f + \sum_{c=1}^{C}\sum_{u=0}^{k-1}\sum_{v=0}^{k-1} W_{f,c,u,v}\, X_{c,\,si+u,\,sj+v}, \qquad H_{\text{out}} = \left\lfloor \frac{H+2p-k}{s} \right\rfloor + 1. \tag{1}$$

Avec $k=5$, $s=1$, $p=0$: $32 \to 28$ (C1) et $14 \to 10$ (C3). Le calcul est réalisé par *im2col* : $Y = \text{cols}\, W^\top + b$.

Pooling. Pour une fenêtre 2×2 de pas 2 :

$$Y^{\text{avg}}_{c,i,j} = \frac{1}{4}\sum_{u,v\in\{0,1\}} X_{c,2i+u,2j+v}, \qquad Y^{\max}_{c,i,j} = \max_{u,v\in\{0,1\}} X_{c,2i+u,2j+v}. \tag{2}$$

Fully connected. $y = xW + b$ avec $W \in \mathbb{R}^{n_{in}\times n_{out}}$. La couche C5 ($16\times 5\times 5 = 400 \to 120$) est équivalente à une convolution 5×5 à 120 filtres ; les deux variantes sont implémentées.

Activations. $\tanh(z)$, $\text{ReLU}(z) = \max(0,z)$ et $\sigma(z) = 1/(1+e^{-z})$, de dérivées $1-\tanh^2(z)$, $\mathbb{1}_{z>0}$ et $\sigma(z)(1-\sigma(z))$.

Perte. Pour les logits $z \in \mathbb{R}^{33}$ et l'étiquette y :

$$p_k = \frac{e^{z_k - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}, \qquad \mathcal{L} = -\frac{1}{N}\sum_{n=1}^{N} \log p^{(n)}_{y_n}, \qquad \frac{\partial \mathcal{L}}{\partial z^{(n)}_k} = \frac{p^{(n)}_k - \mathbb{1}_{k=y_n}}{N}. \tag{3}$$

Rétropropagation. Soit $\delta = \partial\mathcal{L}/\partial Y$ le gradient en sortie d'une couche.

$$\text{Dense :} \quad \frac{\partial \mathcal{L}}{\partial W} = x^\top \delta, \quad \frac{\partial \mathcal{L}}{\partial b} = \sum_n \delta^{(n)}, \quad \frac{\partial \mathcal{L}}{\partial x} = \delta W^\top, \tag{4}$$

$$\text{Conv :} \quad \frac{\partial \mathcal{L}}{\partial W_{f,c,u,v}} = \sum_{n,i,j} \delta_{f,i,j}\, X_{c,si+u,sj+v}, \quad \frac{\partial \mathcal{L}}{\partial b_f} = \sum_{n,i,j} \delta_{f,i,j}, \tag{5}$$

$$\frac{\partial \mathcal{L}}{\partial X_{c,a,b}} = \sum_f \sum_{(i,j,u,v):\, si+u=a,\, sj+v=b} \delta_{f,i,j}\, W_{f,c,u,v}, \tag{6}$$

$$\text{Pool moyen :} \quad \frac{\partial \mathcal{L}}{\partial X_{c,2i+u,2j+v}} = \frac{\delta_{c,i,j}}{4}, \tag{7}$$

$$\text{Pool max :} \quad \delta_{c,i,j} \text{ est transmis uniquement à la position du maximum.} \tag{8}$$

Initialisation. LeCun : $W \sim \mathcal{N}(0, 1/n_{in})$ (par défaut) ; Xavier et He sont aussi disponibles.

2.4 Optimiseurs

Soit $g_t = \nabla_\theta \mathcal{L} + \lambda\theta$ (régularisation L2 sur les poids) et η le taux d'apprentissage.

$$\text{SGD :} \quad \theta_{t+1} = \theta_t - \eta g_t, \tag{9}$$

$$\text{Momentum :} \quad v_{t+1} = \mu v_t - \eta g_t, \quad \theta_{t+1} = \theta_t + v_{t+1}, \tag{10}$$

$$\text{Nesterov :} \quad \theta_{t+1} = \theta_t - \mu v_t + (1+\mu) v_{t+1}, \tag{11}$$

$$\text{RMSprop :} \quad s_t = \rho s_{t-1} + (1-\rho) g_t^2, \quad \theta_{t+1} = \theta_t - \eta \, \frac{g_t}{\sqrt{s_t} + \varepsilon}, \tag{12}$$

$$\text{Adam :} \quad m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{13}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \ \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \tag{14}$$

Valeurs par défaut : $\mu = 0{,}9$, $\rho = 0{,}9$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 10^{-8}$.

2.5 Protocole d'apprentissage

Mini-lots de 64 images ; à chaque époque on calcule loss et accuracy en apprentissage et validation. Les poids retenus sont ceux de la meilleure accuracy de validation ; le test n'est utilisé qu'une seule fois à la fin. Mesures : accuracy, matrice de confusion, précision/rappel/F1 par classe et F1 macro.

Hyperparamètre	Modèle de base
Activation / pooling	tanh / moyenne
Initialisation	LeCun $\mathcal{N}(0, 1/n_{in})$, biais nuls
Optimiseur	Adam, $\eta_0 = 10^{-3}$, $\eta \leftarrow 0{,}95\,\eta$ à chaque époque
Taille de mini-lot	64
Époques (max.) / arrêt anticipé	30 / patience 8
Régularisation	aucune
Graine aléatoire	0
Précision numérique	float32

Tableau 2: Hyperparamètres du modèle de base.

2.6 Vérification de l'implémentation

Les gradients analytiques de toutes les couches sont comparés à des différences finies centrées ($\varepsilon = 10^{-5}$, double précision) : l'erreur relative maximale est $2{,}5\times10^{-7}$ (tanh/moyenne), $1{,}6\times10^{-7}$ (C5 convolutive), $5{,}4 \times 10^{-9}$ (ReLU/max) et $6{,}6 \times 10^{-7}$ (sigmoïde).

3 Résultats

Toutes les expériences sont exécutées sur CPU avec notre code NumPy (environ 10 s par époque pour 19 734 images). Les résultats correspondent à une seule graine ; les écarts inférieurs à environ 0,5 point ne sont donc pas à interpréter comme significatifs.

3.1 Modèle de base

LeNet-5 de base (tanh, pooling moyen, sans régularisation), Adam avec $\eta = 10^{-3}$ décroissant de 5 % par époque, mini-lots de 64, 30 époques.

Mesure	Valeur
Accuracy d'apprentissage (dernière époque)	99,95 %
Meilleure accuracy de validation	98,96 %
Accuracy de test	**99,20 %** (34 erreurs sur 4 224)
Loss de test	0,0290
F1 macro (test)	0,9920

Tableau 3: Résultats du modèle de base sur la base Tifinagh.

Les deux confusions les plus fréquentes sont yaj→yan (7 fois) et yatt→yadd (5 fois) ; yak→yaq apparaît 2 fois et les autres erreurs sont isolées. Plusieurs classes (par exemple yax et yi) sont parfaitement reconnues (F1 = 1).

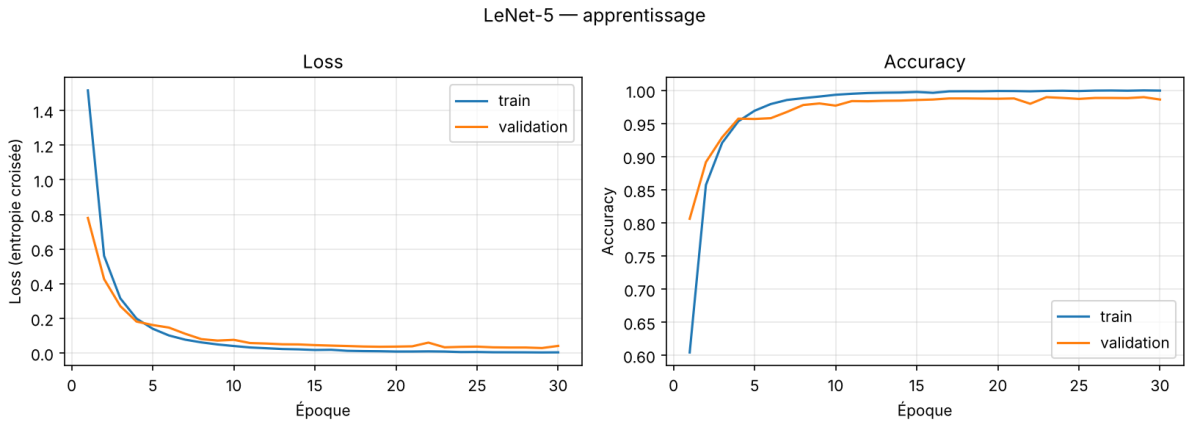


Figure 3: Courbes de loss et d'accuracy (apprentissage et validation), modèle de base.

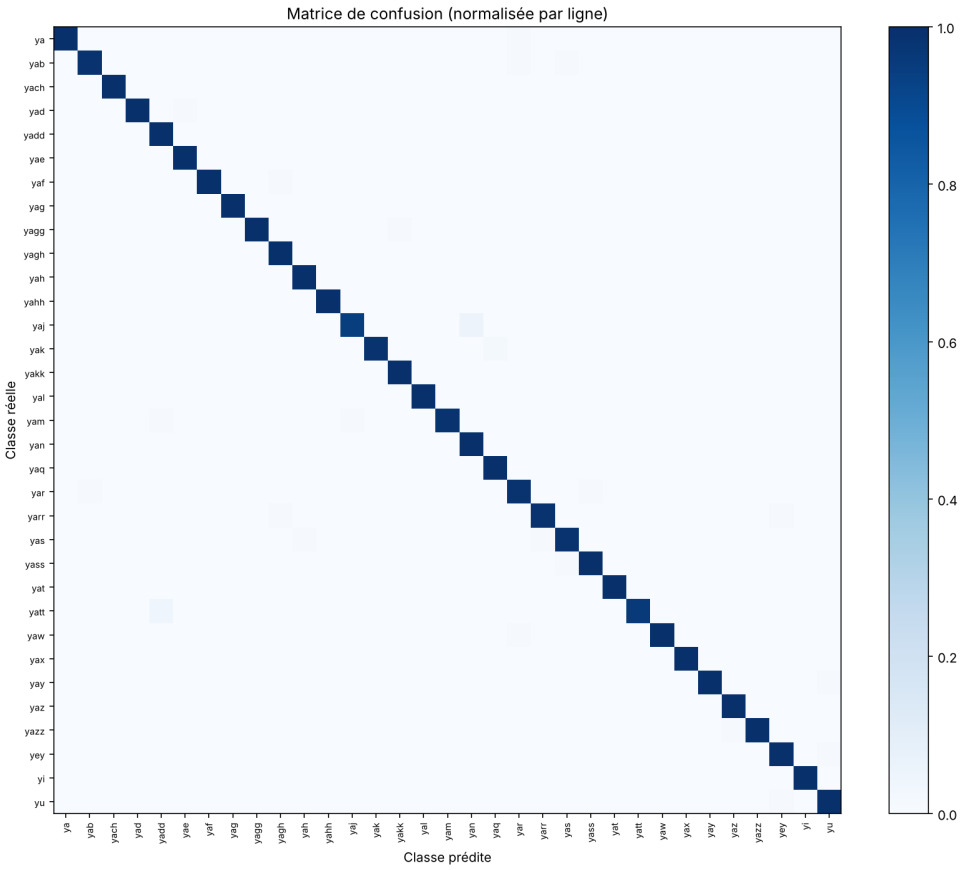


Figure 4: Matrice de confusion normalisée sur le jeu de test.

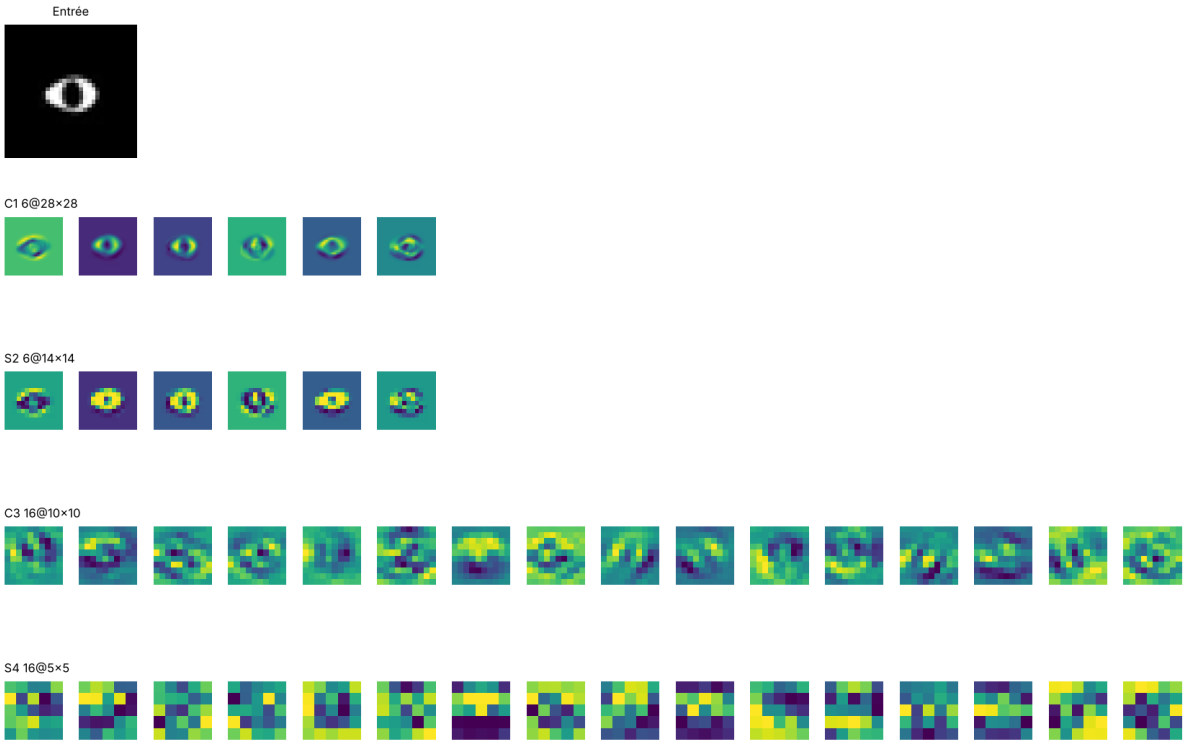


Figure 5: Cartes de caractéristiques de C1, S2, C3 et S4 pour une image de test.

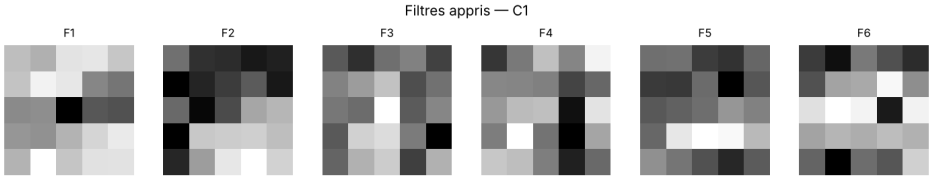


Figure 6: Les six filtres 5×5 appris par C1.

3.2 Comparaison des optimiseurs

Chaque optimiseur entraîne un réseau initialisé de la même façon, pendant 20 époques, avec un taux d'apprentissage constant (aucune décroissance).

Optimiseur	η	Meilleure acc. validation	Acc. test
SGD	0,05	97,75 %	97,87 %
Momentum ($\mu = 0{,}9$)	0,01	98,65 %	98,72 %
RMSprop	10^{-3}	98,79 %	98,72 %
Adam	10^{-3}	98,84 %	98,84 %

Tableau 4: Comparaison des optimiseurs (20 époques, η constant). Les taux d'apprentissage n'ont pas été ajustés finement pour chaque optimiseur.

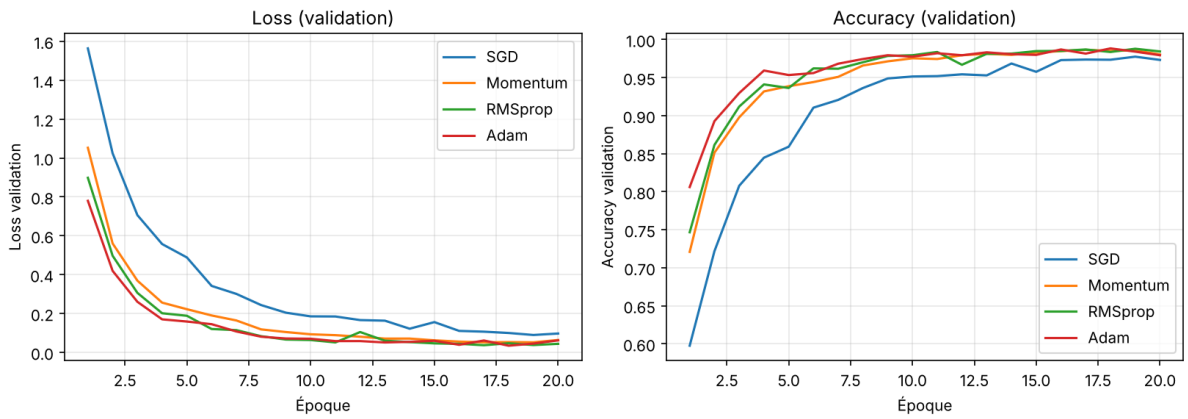


Figure 7: Loss et accuracy de validation par optimiseur.

3.3 Variantes (bonus)

Nous avons testé deux variantes combinant plusieurs modifications. Les protocoles (nombre d'époques, décroissance du taux) diffèrent légèrement de celui du modèle de base ; il s'agit donc d'une comparaison indicative et non d'une ablation contrôlée.

Configuration	Acc. train	Meilleure acc. val.	Acc. test
Base (tanh, pooling moyen)	99,95 %	98,96 %	**99,20 %**
ReLU + max-pool + dropout 0,2 + L2 10^{-4}	99,16 %	98,18 %	98,46 %
ReLU + max-pool + dropout 0,3 + L2 10^{-4} + translations ± 2 px	94,62 %	97,61 %	97,30 %

Tableau 5: Variantes d'amélioration (Adam, $\eta = 10^{-3}$).

4 Discussion

Les optimiseurs adaptatifs (Adam, RMSprop) et le momentum convergent mieux que le SGD simple à nombre d'époques égal (97,9 % contre 98,7–98,8 % en test) ; l'écart entre Adam, RMSprop et Momentum (environ 0,1 point, soit quelques images) est trop faible pour les départager avec une seule graine. Les cartes de C1 montrent des détecteurs de bords orientés, puis C3 combine ces bords en motifs plus complexes, ce qui est cohérent avec la hiérarchie attendue d'un CNN.

Le modèle de base atteint 99,2 % de test, mais son accuracy d'apprentissage (99,95 %) dépasse nettement celle de validation (98,96 %) : un léger surapprentissage est présent. Malgré cela, aucune de nos variantes ne le dépasse. La seconde variante, qui ajoute des translations aléatoires, sous-apprend (94,6 % en apprentissage) ; une explication plausible, que nous n'avons pas testée, est que les caractères de la base sont déjà centrés, de sorte que ces translations éloignent les images d'apprentissage de la distribution de test, et que le dropout réduit encore la capacité d'un réseau déjà petit. Des essais plus longs, avec des translations plus faibles ou d'autres déformations (rotations légères, déformations élastiques), restent à faire.

Les erreurs restantes (34 images) se concentrent sur quelques paires de caractères, notamment yaj/yan et yatt/yadd. Nous n'avons pas examiné les images concernées ; vérifier si ces paires sont visuellement proches ou si certaines étiquettes sont ambiguës serait une étape naturelle. Limites : une seule graine, un seul découpage, des taux d'apprentissage non optimisés par optimiseur, et

une base dont les écrivains peuvent être communs à l'apprentissage et au test (nous n'avons pas pu le vérifier), ce qui peut rendre les scores optimistes.

5 Conclusion

Nous avons fourni une implémentation complète, vérifiée par différences finies, de LeNet-5 pour 33 classes Tifinagh, avec quatre familles d'optimiseurs et les outils de visualisation demandés. Le modèle de base atteint 99,20 % d'accuracy de test avec Adam ; les optimiseurs adaptatifs sont plus rapides que le SGD simple, et les variantes testées n'ont pas amélioré ce score. Les pistes suivantes sont la répétition sur plusieurs graines, un découpage par écrivain si l'information est disponible, la normalisation par lots et des augmentations de données plus adaptées.

Disponibilité du code

Le code source, le notebook `LeNet5_Tifinagh.ipynb`, les scripts `train.py` et `run_all.sh`, les poids appris et tous les résultats cités sont disponibles à l'adresse `https://github.com/Ykhallouki25/Tp-lenet5-tifinagh`. La base AMHCD n'est pas redistribuée dans le dépôt.

Références

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015.